Source: README.md · revision 2420043e1791 · SHA-256 a51a4789ea8c24c926f16bdfa16cc06238bcb0b81c1bd90015cfe7788ae06b5e

# MHFE: Memory-Hard Feistel Encryption for BIP39 Mnemonics

License: CC BY 4.0 Test vectors: CC0 1.0 Specification: 0.6.0 DOI: 10.5281/zenodo.23269085



The penguin lives in the cold, like the backups MHFE is made for. It holds a steel backup with 24 words and waddles from side to side, much as a Feistel network swaps its two halves in every round.

### TL;DR

- **What it does.** MHFE encrypts the original seed phrase, an English BIP39 phrase of 12 to 24 words, with a password. The result is another valid BIP39 phrase, the container phrase, which goes on the usual paper or metal backup. Recovery gives back the exact original seed phrase; the wallet, its funds and any BIP39 passphrase stay as they are.
- **Two suites.** Suite 3 (the default) always produces a 24-word container. For an original phrase of 12 to 21 words, it uses the extra space for a 32- to 128-bit check that detects wrong passwords with high confidence. This works with existing wallets too. A 24-word original leaves no room for that check, but a new 24-word phrase can be generated with an optional 16-bit recovery check, without adding any words. Suite 4 keeps a 12- to 21-word phrase at its original length, but has no built-in password check.
- **What recovery needs.** The container phrase, the password and software that implements the suite. With the default settings nothing else has to be stored. In a rare case the software asks at creation to note the word count of the original seed phrase.
- **Why guessing is slow.** Both suites use twelve consecutive calls to Argon2id, a deliberately slow and memory-hungry function. At the default settings, each call uses 2 GiB of working memory; a full recovery takes about one to two minutes on the reference laptop. For suite 3, the supplement proves in an idealized model that an attacker does no better than testing the most likely passwords at twelve calls each, or eleven if the original seed phrase is also known, apart from the stated terms. With a 24-word original seed phrase and a secret, independent BIP39 passphrase, it bounds the chance of recovering the BIP39 seed by the total probability of the most likely pairs that the budgets allow, counting twelve calls per password and one BIP39 seed computation per pair, plus stated error terms. Security Considerations lists the conditions. Suite 4 is not covered. Attackers can try guesses in parallel. Use a strong password, such as five words chosen independently with dice.

- **A wrong password gives a wallet too.** Even a mistyped password produces a valid BIP39 phrase. Without a matching recovery check or a known wallet reference, a wrong wallet can look just like the right one. A known receiving address or master key fingerprint provides an independent check. The same property lets a second password open a decoy wallet, under the conditions the supplement states.
- **Keep both safe.** A forgotten password loses the wallet. Keep the container phrase private: anyone who has it can try passwords offline. Whoever may have to recover the funds, an heir too, must know that the backup is an MHFE container: entered into an ordinary wallet, it opens an unrelated wallet, normally empty. Keep the original backup until a recovery from the finished new backup has matched a known receiving address of the wallet.
- **Optional extras.** Repair words help restore a damaged container phrase. A sixth password word detects a mistyped word and can recover one forgotten word when its position is known.
- **Status.** Experimental and not independently reviewed: do not use it to protect real funds. The reference implementation, `mhfe`, is a Rust library with a command-line tool and a browser build.

> **MHFE specification version 0.6.0, experimental suites 3 and 4.** Suite 3 is the default; suite 4 provides same-length containers. This version changes the recovery rules: the detected length takes precedence over a stated one, every 24-word reading goes through the 16-bit wallet check, and re-encryption confirms its source with a wallet-identity reference or the owner. It also defines the optional wallet check, repair-word (`MHFE-REPAIR-1`) and password-check-word (`MHFE-PASSWORD-CHECK-1`) profiles and bounded address-path search. Public corpora are in `vectors/suite3/` and `vectors/suite4/`, with provenance, independent replay records and verification limits documented there.

Released as v0.6.0 and archived under DOI 10.5281/zenodo.23269085.

```
BIP: ?
Layer: Applications
Title: Memory-Hard Feistel Encryption for BIP39 Mnemonics
Authors: Sergei Semenov <mr.ssv@protonmail.com>
Status: Draft
Type: Specification
Assigned: ?
License: CC-BY-4.0
Version: 0.6.0
Requires: 39
```

This document is the specification of MHFE suite 3, `MHFE-BIP39-256-EXPERIMENTAL-3`, and of the same-length suite 4, `MHFE-BIP39-LP-EXPERIMENTAL-4`. It uses the format of Bitcoin Improvement Proposals [1], [2]. Detailed analysis, security arguments, cost estimates, related work and research alternatives are collected in the supplement `docs/DESIGN-NOTES.md`.

# Contents


- Abstract
- Motivation
- Conventions and Terminology
- Specification
  - Suite 3: 24-word containers
    - Work factor
    - Application requirements
  - Suite 4: same-length containers
  - Optional source profile: a recovery check for new 24-word phrases
  - Optional repair words: MHFE-REPAIR-1
  - Optional password check word: MHFE-PASSWORD-CHECK-1
- Rationale
- Backward Compatibility
- Security Considerations
- Reference Implementation
- Test Vectors
  - Suite corpora and conformance
  - Optional wallet check: MHFE-WALLET-CHECK-SEED-1
  - Repair words: MHFE-REPAIR-1
  - Password check word: MHFE-PASSWORD-CHECK-1
- Appendix: Suite 2
- AI Assistance and Acknowledgments
- Changelog
- Copyright
- References


# Abstract


MHFE turns an existing 12-, 15-, 18-, 21- or 24-word BIP39 mnemonic [3] into a password-protected container that is itself an ordinary, checksum-valid BIP39 mnemonic: a 24-word container in suite 3, the default, or, in the same-length suite 4, a container of the source's own length for a 12- to 21-word source. With the password, the container is turned back into the exact original seed phrase, so the wallet, its addresses and any BIP39 passphrase stay unchanged. No salt or metadata is stored in the container.

In suite 3 the source is packed into a 256-bit state whose free bits, for a short source, hold a recovery verifier; suite 4 uses the source entropy itself as the state, with no verifier. The state is transformed by a 12-round balanced Feistel permutation. Every round derives its key with Argon2id [4], using 2 GiB of memory by default and a salt of its own, computed from the half of the Feistel state that the round leaves unchanged, the round number and the chosen settings.

In suite 3, accidental salt collisions between independently generated sources are estimated to be negligible under the supplement's model; suite 4's narrower salt diversity is analysed separately. Each round depends on the result of the previous one, so a recovery performs twelve memory-hard calls in sequence. For suite 3, the supplement proves guessing bounds in the random-oracle model that match a twelve-call strategy for an unknown, uniformly generated short source, or an eleven-call strategy for a known source, up to the stated error terms, fractional credit and false-match losses. A 24-word source, like every suite 4 container, has no built-in verifier. A separate optional source profile can provide a statistical recovery check for new 24-word phrases. With a BIP39 passphrase the check needs that passphrase as well; with the empty passphrase it screens MHFE password guesses alone. Without this profile, those unverified readings need external information such as a known address to check a guess. MHFE is designed for cold storage and is experimental: it has not been independently reviewed and must not be used to protect real funds.

# Motivation

**A password to remember and a backup on the usual media.** Without MHFE the phrase itself is the secret: it must be hidden, or learned by heart as 12 to 24 words in their exact order. With MHFE the owner remembers a password instead, and the container, a valid BIP39 phrase of 24 words in suite 3 or of the same length as the original seed phrase in suite 4, goes on the same paper or metal backup, such as a Cryptosteel capsule, which has a fixed capacity for character tiles [5]. Suite 4 suits an existing backup that has room only for the length of the original seed phrase, at the price of having no internal password check. Reading or photographing the container does not directly reveal the original seed phrase, so it needs less secrecy than the phrase itself, but it must not be published: anyone who has it can try passwords offline. The password must therefore be strong and independently generated, for example at least four, better five, words chosen with dice from a published list such as the EFF large wordlist [6]. EFF itself suggests six words; the four or five recommended here follow from the cost of an MHFE guess, and the estimate below shows what such a password costs an attacker.

**Protection of the phrase itself, alongside a BIP39 passphrase.** A BIP39 passphrase changes the wallet derived from a phrase; it does not hide the phrase. Anyone who reads the phrase holds the wallet unless a passphrase is used, and a passphrase is protected only by a fast key derivation. MHFE encrypts the phrase itself with a memory-hard derivation, so each guess of its password is far more expensive, as shown below. The two combine: the original seed phrase is recovered with MHFE first, and the passphrase is then used as before.

**Nothing changes in the wallet.** MHFE encrypts the phrase the user already has. No funds move, and hardware wallets need not support MHFE: after recovery the original seed phrase is entered through their normal recovery procedure.

**Plausible deniability through decoy wallets.** The container is itself a valid BIP39 phrase and can serve as a decoy wallet; nothing in its words shows that MHFE was used. In suite 3, a different MHFE password also yields a valid phrase, read as 24 words, which like every 24-word reading has no internal check and stays unverified unless the user supplies a reference to its wallet; the permutation for that password maps the phrase back to the same container. Its wallet can be funded and used beforehand, providing a working alternative disclosure even when MHFE use is known. The supplement analyses this in a stated model and, under the conditions stated there, bounds the adversary's advantage in telling such a disclosure from an honest one by essentially the probability of guessing the real password; the analysis has not been independently reviewed. A second password does not convince an adversary who knows that the original seed phrase has fewer than 24 words. This paragraph describes suite 3; in suite 4 a decoy keeps the source's length, and the supplement extends the same analysis to suite 4 at every length. A funded decoy of this kind lies outside Theorem 5 of the supplement, whose bound for a 24-word source with an unknown BIP39 passphrase assumes that no other MHFE password opens a funded wallet on the same container.

**Slow on purpose, for cold storage.** A container is created once and recovered rarely, perhaps years later, on a trusted offline computer. Each operation therefore deliberately needs 2 GiB of memory by default. On the reference laptop a recovery takes about one to two minutes natively, and a creation, which ends with a full recovery as its check, about

twice as long; both take longer in a browser without threads. The straightforward attack repeats this work for each password tried. Two optional settings raise the cost further as hardware improves, without changing the format.

**Why not simply a longer passphrase?** A longer passphrase helps and can be combined with MHFE, but it cannot protect the phrase, cannot be added to an existing wallet without moving funds, and each of its guesses is cheap. The difference can be measured in bits of password strength, where each bit doubles the number of guesses an attacker has to try:

- A BIP39 passphrase uses PBKDF2 with 2,048 iterations; the supplement's cost model uses roughly 1.5 million candidates per second, extrapolated from a generic PBKDF2 graphics-card benchmark [7], not a measured complete wallet attack.
- The comparison evaluates each MHFE guess by a full recovery: twelve Argon2id calls with 2 GiB each. The model assumes roughly one candidate per second for the comparison. This MHFE graphics-card rate has not been measured.

Under this full-recovery model, one MHFE guess costs about 1.5 million, or about $2^{20}$, times as much: the same password is about 20 bits more expensive to find, although its entropy does not change. A random word from a 7,776-word dice list carries about 12.9 bits, so this equals roughly one and a half extra words. For example, at those assumed rates, a password of four random words would take about 39 years to find as a BIP39 passphrase, but about 58 million years as an MHFE password, on average. These are order-of-magnitude estimates at the default settings, with their assumptions in the supplement; a higher PIM or memory level adds more.

The problem itself is not new: in 2021 a Bitcoin Stack Exchange question [8] and a bitcoin-dev mailing-list post [9] asked how to encrypt an existing mnemonic into another mnemonic, and the linked prototype reused one AES-CTR keystream for every mnemonic encrypted under the same password [10], although CTR mode requires that counter blocks never repeat under one key [11]. MHFE aims at a reviewed, interoperable answer with a memory-hard KDF and test vectors.

## Conventions and Terminology

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **NOT RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in BCP 14 when, and only when, they appear in all capitals [12], [13].

The original seed phrase, or source, is the wallet's BIP39 mnemonic that MHFE encrypts. The container is the encrypted result; its written words are the container phrase, and the medium that holds them, such as a metal backup, is the backup. A reading is a phrase that recovery derives from the recovered state for one word count; the 24-word reading uses the whole state. A stated length is a word count that the user gives; a detected length is one whose verifier matches. A wallet-identity reference is independent data about the wallet: a known receiving address with its coin, network, address type and derivation path, or its BIP32 master key fingerprint.

Sizes are in bits unless stated otherwise; Argon2id memory is in KiB, as in RFC 9106. Bit offsets count from the most significant bit of the first byte, and bits $Z[8j : 8j + 8]$ form byte $j$. Digests keep their standard byte order; $\mathrm{Trunc}_n(Z)$ is the leftmost $n$ bits of $Z$, read most significant bit first as in BIP39 checksum extraction. $\mathrm{BE32}(v)$ is the unsigned 32-bit big-endian encoding of $v$, $||$ is concatenation and $\oplus$ is bitwise exclusive-or. Implementations MUST NOT use host byte order, hexadecimal text, mnemonic words or string terminators at the suites' cryptographic boundaries. The optional wallet-check profile separately derives a BIP39 seed from mnemonic text exactly as BIP39 specifies.

State sizes and packing notation below describe suite 3; suite 4 overrides them in its section.

| Symbol | Meaning |
|---|---|
| $E$, $\mathrm{ENT}$ | source BIP39 entropy and its length: 128, 160, 192, 224 or 256 |
| $r$, $V_r$ | verifier length $256 - \mathrm{ENT}$ and verifier $\mathrm{Trunc}_r(\mathrm{SHA\text{-}256}(E))$ |
| $X$, $Y$ | 256-bit packed state $E \Vert V_r$ and encrypted state (container entropy) |
| $P_{\mathrm{enc}}$ | normalized UTF-8 password |
| $\mathrm{PIM}$, $\mathrm{MEM}$ | pass multiplier $0 \ldots 1023$ and memory level $0 \ldots 21$; omitted means $0$ |
| $L_i$, $R_i$ | 128-bit halves before round $i$; $L_{12}$, $R_{12}$ are the final halves |
| $S_i$, $K_i$, $M_i$ | round salt (128), Argon2id output (256), round mask (128) |
| $\mathrm{Perm}$, $\mathrm{Perm}^{-1}$ | forward and inverse permutation for one password, PIM and memory level |

# Specification

The procedure below defines suite 3. Suite 4 is defined by its own section, which inherits the shared requirements with the exceptions listed there.

## Suite 3: 24-word containers

### Suite parameters

| Component | Value |
|---|---|
| Suite identifier | `MHFE-BIP39-256-EXPERIMENTAL-3` (ASCII, case-sensitive) |
| Wordlist | English BIP39 wordlist, for source and container |
| State | 256 bits, balanced Feistel with two 128-bit halves |
| Rounds | 12 |
| Salt hash | BLAKE2b with a 256-bit digest [14], truncated to 128 bits |
| KDF | Argon2id version 1.3 (`0x13`), 4 lanes, 256-bit output [4] |
| Argon2id memory | $m(\mathrm{MEM})$ KiB, 2 GiB by default (see Work factor) |
| Argon2id passes | $t(\mathrm{PIM}) = 12 \cdot (\mathrm{PIM} + 1)$, 12 by default |
| Argon2id secret, associated data | empty |
| Round mask | HMAC-SHA-256 [15], [16], truncated to 128 bits |
| Recovery verifier | SHA-256 [15] |

```
SUITE_ID = ASCII("MHFE-BIP39-256-EXPERIMENTAL-3")
DS_SALT  = SUITE_ID || ASCII("/ROUND-SALT")
DS_MASK  = SUITE_ID || ASCII("/ROUND-MASK")
```

The strings have no terminating NUL. These values are frozen; only the PIM and the memory level can be chosen by the user.

The container stores no explicit version field or suite identifier. Its word count selects between suites 3 and 4 within the recovery workflow defined below, but does not distinguish suite 3 from other suites that also use 24 words. Source verifiers, the optional wallet check and wallet-identity references can help assess a candidate recovery, subject to their stated limits. Without such a check, recovering a 24-word source under a wrong 24-word suite can yield a different valid mnemonic without an error signal.

Applications MUST therefore show the suite identifier when they create a container. If the PIM or the memory level differs from its default, applications SHOULD offer the user to record it, because recovery needs exactly the same value. With the default settings and software implementing the intended suite, recovery needs only the container and the password, and the source length is detected automatically, except in the rare case described in step 2 of Creating a container. For long-term storage, users SHOULD also keep an offline copy of a release of compatible software.

Any incompatible change to the geometry, round count, packing, password encoding, salt or mask derivation, Argon2id parameters or the range or mapping of either setting MUST use a new suite identifier and therefore new domain strings. Implementations MUST NOT reuse an identifier for a changed definition or silently substitute one suite for another.

### Password encoding

The MHFE password and the optional BIP39 passphrase are different secrets and MUST NOT be substituted for one another. The password $P$ MUST be a well-formed sequence of Unicode scalar values and is encoded as $P_{\text{enc}} = \text{UTF8}(\text{NFKD}(P))$, using the Normalization Process for Stabilized Strings of UAX #15 [17] with the Unicode 17.0.0 character database: normalization MUST fail if $P$ contains a code point with `General_Category=Unassigned` (`Cn`) in Unicode 17.0.0. This includes the noncharacters, such as U+FDD0; Private Use characters are assigned and accepted. $P$ MUST NOT contain a control character (`General_Category=Cc`, for example U+0000 NUL, U+0009 TAB, U+000A LINE FEED, U+000D CARRIAGE RETURN and U+0085), U+2028 LINE SEPARATOR or U+2029 PARAGRAPH SEPARATOR, and a password that contains one MUST be rejected, never cleaned. The check applies to $P$ before normalization; NFKD maps no other character to any of them, so it holds for $P_{\text{enc}}$ as well. This is a compatibility rule, not a strength rule: excluding controls and line separators avoids common line-ending and input-control problems. Other valid Unicode characters can still be invisible or require suitable input and display support. Implementations MUST NOT apply case folding, trimming or any other transformation. $P_{\text{enc}}$ MUST contain 1 to 1024 bytes; longer or empty results MUST be rejected before any Argon2id call, never truncated. Recovery MUST accept every password that is valid under these rules, whatever strength policy an application applies at creation.

### Reading words

When reading a source mnemonic or a container, implementations SHOULD ignore letter case and extra whitespace and SHOULD accept words abbreviated to their first four letters, because metal backups often store only those letters [5]. Each input word is then resolved as follows: if it equals a word of the English list, it is that word; otherwise, if it has at least four letters and is the beginning of exactly one word, it is that word. Anything else MUST be rejected. The first four letters identify every word of the list uniquely [3], but some three-letter words, such as `act`, also begin longer words, which is why an exact match comes first. Applications SHOULD show the full words they have read back to the user.

### Packing

1. Decode the source with the English wordlist. It MUST have 12, 15, 18, 21 or 24 words and a valid BIP39 checksum.
2. Set $r = 256 - \mathrm{ENT}$. If $r > 0$, set $X = E \| \mathrm{Trunc}_r(\text{SHA-256}(E))$; otherwise $X = E$.

For a short source the first $\mathrm{ENT} \,/\, 32$ bits of $V_r$ are exactly its BIP39 checksum; the rest extend the same hash.

### Permutation

For round index $i$ and a 128-bit right half $R$, $\mathrm{RoundMask}(i, R)$ returns:

$$
\begin{aligned}
S_i &= \mathrm{Trunc}_{128}(\text{BLAKE2b-256}(D_i)),\\
D_i &= \text{DS_SALT} \| \ \mathrm{BE32}(\mathrm{MEM}) \| \ \mathrm{BE32}(\mathrm{PIM}) \| \ \mathrm{BE32}(i) \| R,\\
K_i &= \text{Argon2id}(\text{password} = P_{\text{enc}},\ \text{salt} = S_i,\\
&\qquad \text{memory} = m(\mathrm{MEM})\ \text{KiB},\ \text{passes} = t(\mathrm{PIM}),\ \text{lanes} = 4,\\
&\qquad \text{version} = \texttt{0x13},\ \text{secret} = \text{empty},\ \text{associated data} = \text{empty},\\
&\qquad \text{output} = 32\ \text{bytes}),\\
M_i &= \mathrm{Trunc}_{128}(\text{HMAC-SHA-256}(\text{key} = K_i,\ \text{message} = D'_i)),\\
D'_i &= \text{DS_MASK} \| \ \mathrm{BE32}(\mathrm{MEM}) \| \ \mathrm{BE32}(\mathrm{PIM}) \| \ \mathrm{BE32}(i) \| R.
\end{aligned}
$$

$R$ is the raw 16-byte half; $K_i$ is the HMAC key as is. `BLAKE2b-256` is BLAKE2b with its output length parameter set to 32 bytes, as defined in RFC 7693 [14]; it is not a truncated BLAKE2b-512 digest, which would give different bytes.

$$
\begin{array}{ll}
\text{Forward Perm}: & \text{Inverse Perm}^{-1}: \\
L_0 = X[0:128],\ R_0 = X[128:256] & L_{12} = Y[0:128],\ R_{12} = Y[128:256] \\
\text{for } i = 0, \ldots, 11: & \text{for } i = 11, \ldots, 0: \\
\quad M_i = \mathrm{RoundMask}(i, R_i) & \quad R_i = L_{i+1} \\
\quad L_{i+1} = R_i & \quad M_i = \mathrm{RoundMask}(i, R_i) \\
\quad R_{i+1} = L_i \oplus M_i & \quad L_i = R_{i+1} \oplus M_i \\
Y = L_{12} \| R_{12} & X = L_0 \| R_0
\end{array}
$$

### Creating a container

1. Ask for the password twice and stop if the two entries differ; a mistyped password would make the container unrecoverable with the intended one. Encode the password and validate the PIM and memory level before allocating Argon2id memory.
2. Pack the source. Before the expensive computation, an application SHOULD check whether the packed state passes the verifier of a short-source length other than the source's own. This costs a few SHA-256 computations. If it does, recovery will report that length as a match, so the application SHOULD tell the user to write down the word count of the original seed phrase and keep that note apart from the container, like the password: a note showing fewer than 24 words also shows that a 24-word reading is not the original seed phrase. Nothing is added to the container.
3. Compute $Y = \mathrm{Perm}(X)$.
4. If $Y = X$, refuse the result and ask for a different password, PIM or memory level. This is a fixed point, with probability $2^{-256}$ for a given input in the ideal-permutation model. The check mainly catches a faulty implementation that returns its input unchanged, which would otherwise expose the source.

5. Encode $Y$ with its ordinary 8-bit BIP39 checksum as 24 English words.

6. Decode the words again, compute $\mathrm{Perm}^{-1}$ of the result and compare it with $X$. This MUST be done; it repeats the full computation, but it catches many faults, such as a memory error during the long computation, that would otherwise produce a container that cannot be recovered, and starting from the words also covers their encoding. A mistake shared by the forward and inverse computations survives the round trip; independent test vectors are needed to catch it. On a mismatch the result MUST be discarded and the error reported.

An application MAY show the container to a person while the check runs, so that writing it down overlaps the check. It MUST then mark the container clearly as not yet verified, ask the user not to rely on it until the check ends, and report the outcome: that the container is verified, that it is wrong and must not be used, or, if the check was cancelled, that it remains unverified. A result passed to another program instead of a person MUST NOT be released before the check has passed.

### Recovering a mnemonic

1. Decode the suite 3 container. Anything other than exactly 24 words with a valid checksum is invalid for suite 3 and MUST be rejected before any Argon2id work. A shorter checksum-valid input may be a suite 4 container; it is not necessarily a transcription error.

2. Compute $X = \mathrm{Perm}^{-1}(Y)$.

3. Test the 12-, 15-, 18- and 21-word layouts in every recovery: parse $X$ as $E \| V_r$ for each and compare all $r$ verifier bits. Each layout that matches gives a detected length. The length rules below decide which readings are given.

4. Encode the entropy $E$ of each reading with its BIP39 checksum and its word count.

The user MAY state the length of the original seed phrase. The detected length takes precedence, because a verifier match is far more reliable than memory. With no stated length:

- one length matches: give that reading;
- several lengths match: show each reading with its word count, together with the 24-word reading labelled as not verified; the application MUST NOT pick one silently;
- nothing matches: give the 24-word reading, labelled as not verified.

With a stated length:

- the stated length is among the matches: use it, and name every other length that matches;
- a short length was stated, it does not match, and other lengths do: say so and continue as with no stated length;
- a short length was stated and nothing matches: give no reading, and say that the password or a setting is probably wrong, or that the phrase has 24 words;
- 24 words were stated and a short length matches: show the short readings first and also offer the 24-word reading, labelled as not verified;
- 24 words were stated and nothing matches: give the 24-word reading, labelled as not verified.

Every 24-word reading MUST also go through the wallet check of `MHFE-WALLET-CHECK-SEED-1`, with the BIP39 passphrase the user enters or the empty one. The container does not show whether the phrase was created with that check, so the application cannot skip it. A pass is reported as "passes the 16-bit check". A failure is not an error and does not block the reading; it means something only if the owner knows that the phrase was created with that check.

For certainty, the application MAY offer to check the result against a receiving address the wallet has used, in any recovery. It SHOULD offer this check when the detected length differs from the stated one or when several lengths match. A match identifies the wallet itself.

A 24-word reading MUST be labelled as not verified unless it has matched a wallet-identity reference supplied by the user, as in the rehearsal check below. A verifier match MUST NOT be described as confirmation of the wallet's identity.

### Work factor

$$\begin{aligned} m(\mathrm{MEM}) &= (2 + \mathrm{MEM} \bmod 2) \cdot 2^{20+\lfloor \mathrm{MEM}/2 \rfloor}\ \mathrm{KiB}, & \mathrm{MEM} &\in \{0, \dots, 21\}, \\ t(\mathrm{PIM}) &= 12 \cdot (\mathrm{PIM} + 1), & \mathrm{PIM} &\in \{0, \dots, 1023\}. \end{aligned}$$

The memory level gives 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64 GiB and so on up to 3 TiB at level 21, multiplying by 1.5 and 4/3 in turn so that every value is a whole number of GiB. It MUST be computed with integers. PIM $k$ multiplies the time roughly by $k + 1$, so PIM 1023 turns a recovery of one to two minutes into roughly 17 to 34 hours, and a creation, which includes a recovery as its check, into about twice that.

- Out-of-range values MUST be rejected before memory allocation. An environment that cannot provide the memory MUST fail clearly and MUST NOT reduce it.
- Implementations MUST state which memory levels they support and refuse the others before starting. The reference C implementation of Argon2 limits memory to 2 GiB when pointers are 32 bits wide, so builds for 32-bit WebAssembly support only level 0.
- Applications MUST show both defaults as 0 or as empty fields and SHOULD state the expected time and memory before accepting non-zero values.
- Non-zero values MUST be remembered for recovery, and applications MUST tell the user so when a value other than the default is chosen. Like part of the password, they MAY be kept secret.
- Implementations MUST NOT search settings automatically unless each candidate is checked by the verifier or by a reference as in the rehearsal check below.

### Application requirements

- **Distinct workflow.** A container MUST NOT be passed silently to BIP39 seed derivation, and software MUST NOT guess from the words alone that a mnemonic is a container. The BIP39 passphrase is applied after recovery. No backup of the original seed phrase should be destroyed only because a container exists; the user SHOULD first rehearse recovery and compare a known receiving address of the wallet, derived with the right coin, network, address type and derivation path.
- **Checking the finished backup.** The rehearsal SHOULD read the container from the finished backup rather than from the screen: the creation check covers the words that the application produced, not the copy. If one word is replaced at random, the BIP39 checksum still passes in about one case in 256; for a 24-word source such a container then recovers a different, unverified wallet without any error from the protocol, and the comparison with a known address is what reveals it.

- **Rehearsal check.** Applications SHOULD offer a check that runs a full recovery and reports "matches", "does not match" or "not verified". Besides that result, it reports only facts that reveal no word of the mnemonic: a detected length that differs from the stated one, a pass of the 16-bit wallet check for a 24-word reading, and, on a match with a receiving address, the derivation path described below. It MUST NOT display, copy to the clipboard, export or persist any part of the recovered mnemonic, or report how close a wrong password was. Temporary values needed for the computation are subject to the sensitive-memory requirements below. For a short source the verifier confirms that the password and settings recover a consistent phrase; it does not show that this is the same wallet, and it says nothing about a BIP39 passphrase. The built-in check follows the length rules of Recovering a mnemonic. It reports "matches" when the length rules give exactly one verified reading, at the stated length or at the detected length that takes precedence over it, or when a supplied wallet-identity reference matches. If several lengths match and none of them was stated, it reports the lengths that match. When it gives no reading, it reports "does not match"; when it gives only readings that are not verified, it reports "not verified". For a short source in a suite 3 container it SHOULD ask for the length of the original seed phrase: without it, a wrong password gives an unverified 24-word reading instead of "does not match". For a 24-word source, and whenever the identity of the wallet matters, the user supplies a reference at check time, optionally with the passphrase: a known receiving address with its coin, network, address type and derivation path, or the BIP32 master key fingerprint [18] as a quicker but weaker 32-bit check. Instead of one derivation path, an application MAY search a bounded set of standard address types and derivation paths that it states before the check. On a match with a receiving address it MAY also report the derivation path at which the address was found; this reveals no part of the mnemonic and is shown only on a match. Such a reference SHOULD NOT be stored next to the container. The check belongs on the same trusted offline computer as a recovery.
- **Re-encryption.** Before encrypting a recovered phrase under another password, suite or settings, an application MUST confirm the intended source.
  - *When the verifier is enough.* A short source recovered from a suite 2 or suite 3 container MUST pass that suite's verifier, and its length is the detected one. Where the owner states no length, several lengths match, or the detected length differs from the stated one, the verifier alone does not confirm the source: a 24-word source, or a wrong password, passes the 32-bit verifier of the 21-word layout with probability about $2^{-32}$. The application MUST then confirm the source as for a reading without a verifier. A 21-word source whose stated length matches is accepted on its verifier, with that same $2^{-32}$ risk.
  - *A reading without a verifier.* The application MUST confirm it in one of two ways. The first is a wallet-identity reference, checked as in the rehearsal check: a receiving address or the BIP32 master key fingerprint, with the wallet's BIP39 passphrase when used. The reference MUST exist independently of this recovery: deriving it from the candidate itself, or encrypting the recovered state under the old password and obtaining the old container, is not confirmation. The second, if the owner knows neither, is to show the recovered phrase on a private screen. The owner then enters it into the wallet or compares it word for word with an independently held record of the original seed phrase. The application waits for the owner's explicit confirmation and MUST NOT proceed without that confirmation.
  - *Creating the replacement.* Re-encryption MUST use the complete creation procedure, including its creation check.
  - *Checking the replacement.* An application MUST NOT describe the finished replacement copy as checked, or advise discarding an earlier backup, until recovery from that copy has been rehearsed successfully. The rehearsal uses the source verifier where the replacement has one, and otherwise a wallet-identity reference, which the application MUST have before it creates the replacement. After the source is confirmed, by its verifier or by the owner's explicit confirmation, the application MAY derive a receiving address or the BIP32 master key fingerprint from the confirmed phrase and use it in place of a wallet-identity reference, only for the replacement rehearsal; this does not confirm the source.

- **Preserving derived wallets.** Before replacing a container, applications MUST show every user the same warning, whatever the container. Wallets that other passwords open on the old container depend on its exact words, suite and recovery settings. They do not move to the new container. Keep the old container, its passwords and its settings until the funds of those wallets have been moved. Applications MUST NOT ask which derived wallets exist, request their passwords or require any answer about them before proceeding. Applications MUST also state that re-encryption does not revoke an old container: retained copies still work with their old passwords and settings.
- **Recovery assistance.** Applications MAY try local variants of a half-remembered password, such as other letter case, separators or keyboard layout, each costing one full recovery.
- **Resources.** Recovery SHOULD start only after an explicit user action and SHOULD be cancellable at any moment, not only between rounds, because one round can take hours at a high PIM; stopping the worker or process that runs Argon2 is acceptable. Applications SHOULD check available memory before accepting a memory level: a graphical application SHOULD NOT offer levels the computer cannot provide with a margin, a command-line application SHOULD refuse them with an explanation, and a user choosing a level above 0 SHOULD be reminded that recovery needs a computer with the same amount of memory, and that one must still be available years later.
- **Parallel lanes.** Implementations MUST compute the four Argon2id lanes in parallel where threads are available; otherwise they SHOULD warn that the operation will take longer. The output does not depend on the order of lane computation.
- **Passwords.** Applications SHOULD recommend at least four, better five or more, words chosen with dice from a published list such as the EFF large wordlist [6], and SHOULD warn about weak passwords. They SHOULD advise a different password for each encrypted phrase, used nowhere else; further copies of a backup are exact copies of the same container, with the same password and settings. They SHOULD also say that, after the required NFKD normalization, letter case and the characters between words are significant; a fixed form, such as lowercase words separated by single spaces, is the easiest to reproduce years later.
- **Offline use and no secrets on the network.** Creating and recovering containers for real recovery phrases on a trusted offline computer is strongly RECOMMENDED; a page served from the internet is suitable for demonstration. In every case, implementations MUST NOT send the password, the BIP39 passphrase, the source or recovered mnemonic, or any value derived from them over a network.
- **Sensitive memory.** Implementations SHOULD minimize and erase copies of the entropy, states, salts, password, BIP39 passphrase, BIP39 seed, Argon2id outputs and masks where the runtime allows it. Apart from test vectors made from public inputs, they MUST NOT log, display, export or write to persistent storage any intermediate value, such as states, salts, Argon2id outputs, masks or Argon2id working memory, including in error reports and in files kept to resume an interrupted operation: depending on what leaks and when, a password can then be tested with a single Argon2id call, or with hashing alone, instead of twelve calls.

## Suite 4: same-length containers

Suite 4, `MHFE-BIP39-LP-EXPERIMENTAL-4`, encrypts a 12-, 15-, 18- or 21-word source into a container of the same number of words. A 24-word source has no suite 4 form. Password encoding, reading words, the Argon2id parameters and settings, the work factor and the twelve rounds are those of suite 3. The application requirements apply to both suites, except for the suite 3 rules explicitly excluded below. The rules governing display and release of a container before its creation check completes also apply to suite 4. Suite 3 itself is unchanged.

```
SUITE_ID = ASCII("MHFE-BIP39-LP-EXPERIMENTAL-4")
DS_SALT  = SUITE_ID || ASCII("/ROUND-SALT")
DS_MASK  = SUITE_ID || ASCII("/ROUND-MASK")
```

**State.** The state is the source entropy itself, $X = E$, with $\mathrm{ENT}$ = 128, 160, 192 or 224 bits. No verifier is added. The two Feistel halves have $h = \mathrm{ENT} \,/\, 2$ = 64, 80, 96 or 112 bits; $L_0$ is the first $h$ bits of $X$, $R_0$ the last $h$ bits.

**Round function.** $R$ is the raw $h \,/\, 8$-byte half:

$$
\begin{aligned}
D_i &= \mathrm{DS_SALT} \| \,\mathrm{BE32}(\mathrm{MEM}) \| \,\mathrm{BE32}(\mathrm{PIM}) \\
&\quad \| \,\mathrm{BE32}(\mathrm{ENT}) \| \,\mathrm{BE32}(i) \| R, \\
S_i &= \mathrm{Trunc}_{128}(\text{BLAKE2b-256}(D_i)), \\
K_i &= \text{Argon2id}(P_{\text{enc}}, S_i) \quad \text{with the suite 3 parameters}, \\
D'_i &= \mathrm{DS_MASK} \| \,\mathrm{BE32}(\mathrm{MEM}) \| \,\mathrm{BE32}(\mathrm{PIM}) \\
&\quad \| \,\mathrm{BE32}(\mathrm{ENT}) \| \,\mathrm{BE32}(i) \| R, \\
M_i &= \mathrm{Trunc}_h(\text{HMAC-SHA-256}(\text{key} = K_i,\ \text{message} = D'_i)).
\end{aligned}
$$

$\mathrm{Perm}$ and $\mathrm{Perm}^{-1}$ are those of suite 3 with $h$-bit halves: twelve rounds, $L_{i+1} = R_i$, $R_{i+1} = L_i \oplus M_i$, and $Y = L_{12} \| R_{12}$. For fixed $\mathrm{ENT}$, $\mathrm{MEM}$, $\mathrm{PIM}$ and round index, a salt is a deterministic function of an $h$-bit half and therefore has at most $2^h$ possible values; its encoded length remains 128 bits.

**Choosing the suite.** A 12-, 15-, 18- or 21-word source can be encrypted under suite 3, giving a 24-word container whose recovery has an internal check, or under suite 4, giving a container of the same length without one. Suite 3 is the default for every source length; an application MUST use suite 4 only when the user has chosen it for that container. It MUST show the consequences of the choice and, after creation, the suite identifier.

**Creating a container.** Steps 1, 3, 4 and 6 of suite 3 apply. The source MUST be 12, 15, 18 or 21 words with a valid BIP39 checksum. A fixed point $Y = X$ is refused as in suite 3; for a given input it has probability $2^{-\mathrm{ENT}}$ in the ideal-permutation model. $Y$ is encoded with its own BIP39 checksum as a container of the source's word count. The check of step 6 confirms that the container was created correctly; a later recovery is still unconfirmed without a reference.

**Recovering a mnemonic.** Invalid words, an invalid checksum and an unsupported length MUST be rejected before any Argon2id work. The container's word count selects the suite, as described under *Choosing the suite and the length during recovery* below. A suite 4 container has the same length as its source and looks exactly like an ordinary wallet phrase, so nothing on the backup shows which one is the original seed phrase: the user must keep track of which backup is which, and importing the container into a wallet gives an unrelated, valid wallet. Compute $E = \mathrm{Perm}^{-1}(Y)$ and encode it with its BIP39 checksum as a phrase of the same word count. Every password gives a valid phrase, so the result MUST be labelled as not verified unless it has matched a wallet-identity reference supplied by the user, as in the rehearsal check.

**Choosing the suite and the length during recovery.** In a recovery workflow for suites 3 and 4, the application tells the two suites apart only by the container's word count, because nothing else in the phrase shows it: applications MUST select suite 4 for a 12-, 15-, 18- or 21-word container and suite 3 for a 24-word container. Word count does not establish that a phrase is an MHFE container or identify suites outside this workflow; recovering a suite 2 container requires its explicit selection. A suite 4 container always gives a phrase of its own length, so the application asks the user for no length. It asks for the length of the original seed phrase only for a 24-word container, under the length rules of Recovering a mnemonic. Where a length or a suite is supplied together with the container, for example through a programming interface, the application admits to recovery only the containers that fit it; this does not confirm that a container is the right one. Every other container is rejected by its word count, as are invalid words and checksums, before any Argon2id work, and a rejected container MUST NOT be recovered under another suite:

| Supplied with the container | The application admits to recovery |
|---|---|
| A length of $n$ = 12, 15, 18 or 21 words | a suite 4 container of exactly $n$ words, decrypted to an $n$-word mnemonic, or a suite 3 container of 24 words, under the length rules of Recovering a mnemonic |
| A length of 24 words | a suite 3 container of 24 words only |
| Suite 3 | 24-word containers only; every 12- to 21-word input is rejected |
| Suite 4 | 12- to 21-word containers only; a 24-word input is rejected |

When both a length and a suite are supplied, both restrictions apply. For example, with a length of 18 words supplied, a container of 12, 15 or 21 words is rejected.

**Rules of suite 3 that do not apply.** Suite 3's packing, length detection with its rules for a stated length, and short-source verifier checks do not apply to suite 4: recovery keeps the container's word count and needs an external wallet-identity reference for confirmation. A short container has a BIP39 checksum of 4, 5, 6 or 7 bits, so a word replaced at random during copying still passes the checksum in about one case in 16, 32, 64 or 128, not one in 256. Applications SHOULD therefore stress for suite 4 that the rehearsal check reads the container from the finished backup, not from the screen.

**Security.** Suite 4 uses the same Argon2id parameters as suite 3. A full recovery performs twelve calls, and the supplement's known-pair filter uses eleven in either suite. The narrower state does not reduce the cost of these procedures. As with suite 3's 24-word reading, recovery alone provides no internal password confirmation.

These procedure costs are not proven lower bounds for suite 4: the supplement's proofs for suite 3 do not carry over to its narrower halves, and suite 4 also needs its own analysis of salt reuse and multi-target attacks. Equal recovery work factors alone do not establish equivalent security. For a 12-word source a round salt takes at most $2^{64}$ values for fixed settings. RFC 9106 permits a 64-bit salt length under space constraints [4]; this provides context, not a security justification, because suite 4's salts are derived from a state half. The supplement analyses what the narrower state changes and extends the deniability analysis to suite 4. Applications MUST tell the user that a mistyped password or setting is not detected and recovers a different valid wallet, and that the container reveals the source's word count.

## Optional source profile: a recovery check for new 24-word phrases

An application MAY offer the source-generation profile `MHFE-WALLET-CHECK-SEED-1` as an explicit choice when creating a new 24-word wallet. Applications MUST explain the benefits, false matches, entropy-conditioning and deniability costs before the owner chooses an ordinary random phrase or a phrase selected to pass the check below. An application MAY offer the profile only with a nonempty BIP39 passphrase; it is not required to offer the empty-passphrase option. The profile changes neither suite identifier nor packing, encryption, decryption or the container's word count. A 24-word source still uses suite 3. No check words, salt or profile metadata are added to the container. Existing wallets cannot generally acquire this property without changing their phrase; an existing phrase may already pass by chance.

**Check definition.** Let $E$ be the 256-bit source entropy and $M(E)$ its checksum-valid English BIP39 mnemonic, in lower case with exactly one ASCII space between words. Let $Q$ be the wallet's BIP39 passphrase, or the empty string for a wallet without one. The seed is derived exactly as in BIP39 [3], independently of MHFE password encoding:

$$
\begin{aligned}
\text{seed} = \text{PBKDF2-HMAC-SHA512}(&\\
&\text{password} = \text{UTF8}(\text{NFKD}(M(E))),\\
&\text{salt} = \text{ASCII}(\texttt{mnemonic}) \,\|\, \text{UTF8}(\text{NFKD}(Q)),\\
&\text{iterations} = 2048,\ \text{output} = 512 \text{ bits}),\\
T = \text{SHA-256}(&\text{ASCII}(\texttt{MHFE-WALLET-CHECK-SEED-1})\\
&\| \text{BE32}(256) \| \text{seed}),\\
\text{passes if and only if } &\ \text{Trunc}_{16}(T) = 0^{16}.
\end{aligned}
$$

The domain tag is exactly 24 ASCII characters, without a terminating zero. It is a public constant included in the hash, not a password or a field stored on the backup. $\text{BE32}(256)$ is the 32-bit big-endian integer 256, in hexadecimal `00 00 01 00`. Neither the first bits of $E$ nor those of seed are set to zero: the condition is on their derived digest $T$.

**Generation and recovery.** Hold $Q$ fixed and draw fresh, uniformly random $E$ from a cryptographic generator until the check passes; then create the suite 3 container normally, including its mandatory creation check. Recovery follows the suite 3 procedure, which evaluates this check for every 24-word reading, whether or not the application offers the profile at creation: see Recovering a mnemonic. A pass does not replace a wallet-identity reference: without one, a 24-word reading stays labelled as not verified.

**Trade-offs.** In the idealized model, generation takes about $2^{16}$ BIP39 seed computations on average, leaves approximately 240 bits of source entropy for a fixed $Q$, and an incorrect recovery passes with probability approximately $2^{-16}$, compared with $2^{-8}$ for random words passing the 8-bit BIP39 checksum of a 24-word phrase. These are different events: the BIP39 checksum checks the recorded words; an incorrect MHFE password still recovers a phrase with a freshly computed, valid BIP39 checksum. This 16-bit check is weaker than the 32- to 128-bit verifiers of short sources in suite 3, and like them it is not authentication. With a secret, independent BIP39 passphrase, this check alone tests mnemonic/passphrase pairs, not the MHFE password separately. With the empty passphrase, it also lets an attacker screen MHFE password guesses. Changing the BIP39 passphrase normally causes the check to fail, but another passphrase can pass by chance or after a search. The deniability theorems for uniformly random sources do not automatically cover this conditioned distribution; the supplement analyses guessing costs, decoy preparation and the limits of the check. Other source lengths and entropy-only checks discussed there are not part of this profile.

**Recommended use.** A strong, independently generated BIP39 passphrase is the recommended way to use this optional check. This preserves the stronger protection of testing MHFE password and passphrase pairs: the check requires both secrets. The owner pays the seed-search cost once at creation, and a later check needs only one BIP39 seed computation after MHFE recovery; an attacker repeats the expensive recovery for MHFE password candidates and tests passphrase candidates for each. This protection comes from the structure of the scheme, not from the check.

To preserve this combination, all funds stay under that passphrase, the wallet with the empty passphrase remains unused, and no separate reference identifies the original seed phrase without the passphrase.

Theorem 5 of the supplement proves this protection in the random-oracle model.

- **Bound.** Every attack's chance of obtaining the BIP39 seed is at most the total probability of the most likely password and passphrase pairs that its budgets allow, at twelve Argon2id calls per tested password and one BIP39 seed computation per tested pair, plus explicit terms.
- **Against an attacker who knows the wallet's addresses or its fingerprint**, this is, apart from the stated error terms, the bound of an ordinary 24-word phrase with the same passphrase.
- **Against an attacker without wallet data**, the bound still holds, but the check is a 16-bit pair filter that an ordinary phrase does not give.

- **Conditions:**
  - the three conditions above;
  - secrets chosen independently of each other;
  - a generation that draws independent uniform phrases, takes a passing one by the order and outcomes of its trials alone, and reveals nothing about its trials, their number or its duration, as the sequential procedure above does;
  - no funded wallet that another MHFE password opens on the same container. The proof does not cover such a decoy.
- **With the empty or an independently known passphrase**, the theorem also bounds password recovery. The wallet reference included in the experiment is allowed; additional phrase-identifying data and funded openings under other MHFE passwords remain outside the model. The generation conditions still apply.

The profile also defines an empty-passphrase form for applications that offer it and owners who choose its different trade-offs; it does not provide this two-secret protection.

Incompatible changes to this profile's definition require a distinct profile identifier, without changing suite 3.

Public vectors are listed under Test Vectors.

## Optional repair words: MHFE-REPAIR-1

An application MAY offer repair words for a finished container: $k$ extra English BIP39 words, written on a separate card, that let an application repair unreadable or miscopied container words without the password and without any Argon2id work. The profile changes neither suite. The container stays as it is, and recovery from an intact container phrase does not use the card. The card is released only after the final container's creation check has passed; its parity may be computed internally beforehand. It belongs to that container only: after a re-encryption, the new container needs a new card. The card SHOULD carry the profile name and number its words in order, for example `1/8` to `8/8`, so that a missing card word becomes an unreadable word at a known position.

**Code.** The symbols are elements of $\mathrm{GF}(2^{11})$ defined by the primitive polynomial $x^{11} + x^2 + 1$ with $\alpha = x$; an element is a BIP39 word number from 0 to 2047, whose bit $i$ is the coefficient of $x^i$. For a container of $n$ words, with $n$ = 12, 15, 18, 21 or 24, and word numbers $d_0$ to $d_{n-1}$, first word first:

$$
\begin{aligned}
m(x) &= d_0 x^{n-1} + d_1 x^{n-2} + \cdots + d_{n-1}, \\
g(x) &= (x + \alpha^1)(x + \alpha^2) \cdots (x + \alpha^k), \quad k \in \{2, 4, 6, 8\}, \\
r(x) &= m(x) x^k \bmod g(x) = r_{k-1} x^{k-1} + \cdots + r_0.
\end{aligned}
$$

$$
\text{Repair words, in order: } r_{k-1}, r_{k-2}, \ldots, r_0.
$$

The container phrase followed by the card is then a codeword of a shortened Reed-Solomon code: read as a polynomial in the same way, it vanishes at $\alpha^1$ to $\alpha^k$. The coefficients of the generator polynomials, highest degree first, are:

$$
\begin{gathered}
g_2 : 1, 6, 8 \\
g_4 : 1, 30, 216, 960, 1024 \\
g_6 : 1, 126, 1181, 1719, 2029, 1077, 1034 \\
g_8 : 1, 510, 1509, 1770, 1837, 850, 1339, 600, 680
\end{gathered}
$$

**Repair.** The code corrects any combination of $e$ wrong words at unknown places and $s$ unreadable words at known places with $2e + s \le k$, counting words of the container phrase and of the card alike. Each wrong word consumes two parity symbols; this profile offers even values of $k$. Container and card words are read as under Reading words; a token that does not resolve to a word there counts as unreadable instead of being rejected. The code does not guarantee to repair insertions or deletions that shift later words to other positions; a missing word marked `?` at its own position is an ordinary unreadable word and is repaired. An application MUST accept a repaired container only if it passes its BIP39 checksum, MUST show every repaired word with its position and with what was read there or that it was unreadable, before any recovery, and MUST NOT correct silently. Beyond that bound a decoder can fail or return a wrong container. The BIP39 checksum may detect a wrong result, but does not guarantee detection. An application that offers repair beyond the bound MUST label its result as a guess. The decoder cannot always recognize that the actual damage exceeds the bound or that the card belongs to another container phrase. A successful repair and checksum do not authenticate the result; the owner still needs to rehearse recovery against the usual source verifier or an independent wallet-identity reference.

**Storage.** The card is computed from the container, so it adds nothing for someone who holds the whole container phrase. For uniformly random container entropy, treating the BIP39 checksum as an independent hash constraint, a card on its own leaves about $2^{11(n-k)-n/3}$ checksum-valid candidate containers, where $n$ is the number of container words, $k$ the number of repair words and $n \,/\, 3$ the container checksum length: for a 24-word container even eight repair words leave about $2^{168}$, but for a 12-word suite 4 container eight words leave only about $2^{40}$. These are candidate-count estimates, not the cost of identifying the true container or guessing its MHFE password. In the shorter case, enumerating the candidates is a realistic additional attack surface. Together with a damaged or partial copy of the container phrase, the card completes that copy for anyone, and a complete container is what a password guesser needs. With all $k$ repair words correct, the remaining container words read correctly and every gap marked at its position, $n - k$ container words are enough; with wrong words at unknown positions, the bound $2e + s \le k$ applies. Applications SHOULD tell the owner to keep the card apart from the container phrase and to guard it like the container phrase. Written next to the container phrase, the card would also show that the backup is a special one.

Public vectors are listed under Test Vectors.

## Optional password check word: MHFE-PASSWORD-CHECK-1

An application MAY generate passwords with a check word: five words drawn independently and uniformly from the EFF large wordlist [6] and a sixth word computed from them. The whole six-word string is the MHFE password, and suites 3 and 4 process it like any other password; the check word only lets an application notice and repair typing errors before any Argon2id work.

**Definition.** The list is the EFF large wordlist file `eff_large_wordlist.txt` [6]: 7,776 lines of the form `<dice><TAB><word>` with LF line endings, SHA-256 `addd35536511597a02fa0a9ff1e5284677b8883b83e986e43f15a3db996b903e`. The index of a word is its line number minus one, so five dice rolls $r_1$ to $r_5$ give the index $(r_1 - 1) \cdot 6^4 + (r_2 - 1) \cdot 6^3 + (r_3 - 1) \cdot 6^2 + (r_4 - 1) \cdot 6 + (r_5 - 1)$, from 0 for `11111` to 7,775 for `66666`. Its words, including the four with a hyphen, `drop-down`, `felt-tip`, `t-shirt` and `yo-yo`, are used as written. For the indexes $d_1$ to $d_5$ of the five drawn words:

$$
\begin{aligned}
c &= (d_1 + 5d_2 + 7d_3 + 11d_4 + 13d_5) \bmod 7776, \\
\mathrm{SP} &= \text{one ASCII space (U+0020)}, \\
\text{password} &= \mathrm{word}(d_1) \| \mathrm{SP} \| \mathrm{word}(d_2) \\
&\quad \| \mathrm{SP} \| \mathrm{word}(d_3) \| \mathrm{SP} \| \mathrm{word}(d_4) \\
&\quad \| \mathrm{SP} \| \mathrm{word}(d_5) \| \mathrm{SP} \| \mathrm{word}(c).
\end{aligned}
$$

In this profile's canonical form, the words are lower case and separated by single ASCII spaces (U+0020), with the check word last and no other characters; other MHFE passwords are not restricted by this form.

**Properties.** Every coefficient is coprime to 7,776, so any single word replaced by a different list word is detected, and one erased word at a known position, the check word included, is recovered uniquely. A single replaced word cannot be located: each of the six positions admits exactly one repair, so a detected error leaves six candidates, not the right position. Transpositions are not always detected; the supplement gives their rates. The five drawn words carry about 64.6 bits; the check word adds no randomness and must stay as secret as the rest of the password. If an attacker learns only the check word, the remaining uncertainty falls by $\log_2(7776)$, about 12.9 bits, to about 51.7 bits. Keeping all six words secret retains the original 64.6 bits. A matching check word shows only that the words fit together, not that the password opens the intended container.

**Checking.** An application that knows the password was made with this profile splits the typed string at U+0020 spaces and compares each token with the list exactly as written, hyphens included. The Reading words rules for mnemonics do not apply, because the list holds both `yo-yo` and `yoyo` and many of its words share their first four letters. Checking and repair require exactly six token positions. A token not on the list, including a placeholder `?`, is an erased word at that position. Unique erasure repair requires exactly one erased word and five known list words. More than one erased word does not have a unique repair under this rule. Omitting a word without a placeholder loses its position and shifts later tokens; it is not the same as an erasure at a known position. An application MUST NOT present an input with a different token count as a uniquely repairable six-position password.

An application MUST show every proposed repair or rewrite and obtain the user's confirmation before any Argon2id work, and MUST NOT correct silently. A failing check gives a warning and MUST NOT by itself block recovery with an otherwise valid MHFE password as entered, since the profile is optional and the container does not record it. The general password-encoding rules, including the forbidden characters and normalized byte-length limit, still apply. When the real password follows this profile, a password that may be disclosed for the same container SHOULD be generated in the same way, because the deniability analysis assumes that a decoy is drawn like the real password.

Public vectors are listed under Test Vectors.

# Rationale

The design decisions are explained here for suite 3; suite 4's differences are stated in its own section. The supplement develops the security arguments, cost models and research alternatives in detail.

**Why a Feistel network with state-derived salts?** Encrypting 256 bits without stored data is easy with one key $\mathrm{Argon2id}(\mathrm{password}, \mathrm{constant})$, but then the salt is the same for everyone and a dictionary computed once attacks every container at once. The only material unique to a container is the container itself, so the salt must come from the encrypted state and be recomputable during decryption. A Feistel network keeps one half unchanged in each round, which is exactly what allows a different salt in every round. SLIP-0039 uses the same idea with PBKDF2 [19]. MHFE does not claim conformance to NIST SP 800-132, which requires a randomly generated salt part of at least 128 bits for PBKDF2 [20]: its salts are derived from the state so that the container keeps its fixed size without stored data. Their suitability is examined in the supplement; the 128-bit length and the estimate of accidental collisions do not by themselves make them equivalent to independently generated salts. An unbalanced Feistel network [21], Lai-Massey [22], [23] and swap-or-not [24] could also work. Their published bounds call, respectively, for more rounds when rounds are few, a mixing step between rounds, or hundreds of rounds; these are sufficient conditions rather than proven minimums, but every extra round costs an Argon2id call.

**Why Argon2id?** It is specified in RFC 9106, an Informational RFC of the IRTF Crypto Forum Research Group, which names Argon2id its primary variant [4]. Argon2 won the Password Hashing Competition, an open competition with 24 candidates [25], [26], and Argon2id is the first choice of the OWASP password-storage recommendations [27]. During

the first half of its first pass Argon2id accesses memory independently of the password, which resists cache-timing side channels; afterwards it uses password-dependent access, which raises the cost of trading memory for time. Its memory, passes and parallelism are separate parameters, while PBKDF2 and bcrypt need little memory. Several independent implementations exist, including the reference code, OpenSSL and RustCrypto, which matters for a backup that may have to be decrypted many years later. Its optional secret and associated data, which RFC 9106 defines, stay empty: many Argon2 interfaces take only a password and a salt, MHFE defines no additional secret, and the suite, the settings and the round index already enter through the salt and mask messages.

**Why Argon2id in every round, and why twelve rounds?** Each round's salt depends on the previous round, so a recovery makes the twelve Argon2id calls in sequence. In the random-oracle model the supplement proves suite 3 guessing bounds matching a twelve-call strategy for a uniformly generated, otherwise unknown source, or an eleven-call strategy for a known source and container, up to the stated error terms, fractional credit and false matches. The unknown source is checked by its verifier or against wallet data with the empty or a known BIP39 passphrase. The known-pair shortcut skips one round, a discount of one twelfth. Twelve rounds give natural progress and cancellation points, and exceed the round counts for which Patarin proves strong bounds for ideal Feistel networks [28], [29], building on the construction of Luby and Rackoff [30]. Beyond the cost of a guess, twelve rounds are a conservative choice rather than a proven margin: MHFE's round functions are not independent random functions. The round count and the Argon2id parameters jointly determine the waiting time, so the time budget alone does not justify twelve rounds.

**Why BLAKE2b for salts and HMAC-SHA-256 for masks?** The salt is computed without a key from the half of the state that the round leaves unchanged, so a fast unkeyed hash suffices. Only the first salt of a recovery can be computed from the container, and the first salt of an encryption from the original seed phrase; the other salts depend on the password and are as sensitive as the states. BLAKE2b is used so that salt derivation shares no function with the SHA-256 verifier. The mask must depend on the secret Argon2id output, so it needs a keyed function, and HMAC-SHA-256 is the standard choice. The mask also uses SHA-256, but only inside HMAC keyed with that secret output and a distinct domain string. The domain strings also separate salts from masks.

**Why 2 GiB and twelve passes?** RFC 9106's first recommended option is Argon2id with 2 GiB, four lanes and one pass, and its selection procedure takes the largest affordable memory and then the largest number of passes that fits the available time [4]. The twelve passes are this project's choice under that procedure, because a cold-storage operation can afford several seconds per round. For the same time, more memory would, in an area-time cost model and for an attacker limited by memory capacity, cost an attacker more than more passes, but 2 GiB is the most that the reference C implementation of Argon2 accepts in a 32-bit WebAssembly build, a limit of that implementation rather than of the WebAssembly address space, and the most that many computers can spare, so the rest of the time goes into passes. On a 2022 mid-range laptop (Intel Core i7-1260P) one such call took about 5 seconds with the reference C implementation and about 10 seconds with OpenSSL, so a recovery takes about one to two minutes and a creation with its check about twice as long. What a guess costs gives the attacker cost model and its assumptions.

**Why two settings, and why these limits?** Memory must fit the computer that will perform recovery, possibly years later, while waiting time is a matter of patience; one number could not serve both. PIM stops at 1023 because a factor of 1,024 adds only ten bits while recovery takes roughly 17 to 34 hours on the reference laptop; one more dice word adds 12.9 bits for free. The memory level stops at 21 because level 22 would exceed the Argon2id limit of $2^{32} - 1$ KiB. An owner may keep a setting secret, but that helps only to the extent that its value is unpredictable to the attacker, and it gives no fixed additional margin; a forgotten setting must also be searched during recovery. A strong, independently generated password remains the primary protection. The parameter rationale estimates the effect of a secret PIM under stated assumptions.

**Why a verifier for short sources and none for 24 words?** A short source leaves free bits in the 256-bit state, and filling them with a hash of the source adds no storage and lets recovery screen the password and detect a likely length, subject to false matches. A 24-word source has no free bits. Both behaviours are useful, and the user chooses by the

length of the original seed phrase:

| Original seed phrase | Recovery | With a BIP39 passphrase |
|---|---|---|
| 12 to 21 words | screens the password and detects a likely length; the 32-bit verifier of a 21-word source admits false matches in large searches | the costs add up only while work on false matches is negligible |
| 24 words | cannot confirm; a wrong password gives another valid wallet | independent secrets require a search over pairs if no separate check identifies the original seed phrase |

For a newly generated 24-word source, the optional source profile adds a statistical check without additional storage. This is a property of source generation, separate from the built-in verifier described in the table.

A 24-word original seed phrase gives the strongest combination when the two secrets are independent and no separate check identifies that phrase. In suite 3, a 12- to 21-word original seed phrase provides an internal recovery check, but false matches may require additional passphrase searches, especially with a 21-word source. Without a passphrase, a funded 24-word wallet lets a guesser confirm a password through the blockchain anyway. The composition analysis gives the search costs of both cases and explains when work on false matches dominates. For 24 words, Theorem 5 of the supplement bounds seed-recovery probability for given computational budgets in the random-oracle model, with explicit error terms and the conditions listed in the profile's recommended use; for 12 to 21 words the costs are estimates rather than lower bounds. Related or reused secrets lose these gains.

**Why is the final word not preserved?** A 24-word original seed phrase has no verifier. One conceivable aid would be a container that ends with the same last word as the original seed phrase: that word carries the phrase's 8-bit checksum, so the owner could at least recognise which container phrase belongs to which wallet. Such a container can be found by cycle walking [31], that is, by applying the permutation again and again until the last word matches. This needs about 2,048 permutations on average, each as long as a recovery, which with the suite 3 parameters means about one and a half to three days per recovery on average at one to two minutes per permutation, about twice that for a creation with its check, and longer in a browser without threads. It would also reveal the last word, which is three entropy bits directly and about 11 bits of information about the source in total, and would still not confirm the password, because a wrong password also walks to a phrase with the same last word. The idea is therefore not used; it is recorded as research.

**Why NFKD and Unicode 17.0.0?** NFKD is the normalization that BIP39 applies to mnemonics and passphrases [3], and it makes compatible variants, such as full-width and ordinary Latin letters, give the same password. BIP39 support in a library is not enough, however: MHFE also uses the process for stabilized strings with a pinned Unicode version and rejects the characters listed under Password encoding. Words are read forgivingly because each resolves to one entry of a fixed list, so nothing is lost. A password is free text, in which letter case and spacing can be part of what the owner chose, and any further normalization would have to stay frozen with the suite; it is therefore used as entered, apart from NFKD. Normalization of assigned characters never changes, so a version pin only decides which new characters are rejected. Unicode 17.0.0 is implemented by current libraries, including Rust's. Suite 2 had pinned Unicode 18.0.0, the newest version at the time, for which Rust had no normalization tables yet, so its reference implementation accepted only ASCII passwords; suite 3 deliberately returns to 17.0.0 so that full Unicode passwords work with existing libraries.

**Why only the English wordlist?** Suite 3 reads sources and writes containers with the English BIP39 wordlist only. BIP39 itself recommends the same: its Wordlists section strongly discourages other wordlists for generating mnemonics, because the vast majority of BIP39 wallets support only the English one [3]. BIP39 also derives the seed from the words of the mnemonic, not from its entropy, so the same entropy written with another wordlist gives a different wallet: a phrase in another language cannot be accepted by re-encoding it in English words, and recovery could restore the original language only if the user remembered it as one more setting. The container has no room to

record a wordlist, and the rule for four-letter abbreviations under Reading words is defined and checked for the English list. Support for other wordlists would need its own suite identifier that fixes the wordlist. Other seed formats, such as Electrum seeds and SLIP-0039 shares, are outside this specification.

**Related work.** SLIP-0039 [19] encrypts a master secret with a Feistel network over PBKDF2 but is a secret-sharing format. BIP38 [32] protects single private keys with scrypt in an expanded record. MnemonicCrypt [33], Mnemonikey [34] and pktseed [35] add salts, versions or other fields and change the mnemonic length or wordlist; seed-otp [36] needs a second secret as long as the mnemonic; Seedshift, bip39_obfuscator and BIP39Colors [37]-[39] offer obfuscation, not memory-hard encryption. Monero's seed offset passphrase [40], Polyseed [41] and the seed-encrypt tool [42] keep the length of a seed under a password, but with no salt or one salt shared by every user; the supplement compares them. Polyseed also treats its mnemonic as a polynomial over $\mathrm{GF}(2048)$ with one Reed-Solomon check word [41]; the optional repair words (MHFE-REPAIR-1) apply the same kind of code to a finished container, with 2, 4, 6 or 8 words on a separate card. For a 24-word source MHFE has the structure of honey encryption [43] with a uniform message model, and its plausible deniability is modeled on deniable encryption [44] but is narrower: the decoy phrase is whatever a chosen decoy password recovers, not a phrase chosen freely; both are analysed in the supplement.

**Can an optional generator select a mnemonic for a recovery check?** BIP39 defines the entropy length, checksum and word encoding, and separates mnemonic generation from conversion to a seed; it contains no explicit prohibition on rejection sampling [3]. Drawing random candidates and retaining only those that pass a predicate preserves the standard mnemonic format and seed derivation, but adds a distinct generation profile with a restricted entropy distribution. BIP39 compatibility is not a security endorsement. The optional source profile defines one such check; the supplement analyses its trade-offs and alternatives. It is not part of the suites' encryption algorithm.

Electrum provides a precedent for selecting generated mnemonics by a hash prefix: its seed-version system enumerates a nonce and rehashes the phrase until $\mathrm{HMAC\text{-}SHA\text{-}512}(\mathrm{key} = \texttt{"Seed version"},\ \mathrm{message} = \mathrm{normalized\ phrase})$ has the required version prefix [45]. Its documentation also discusses the effect of this prefix on the attack cost of key stretching. Electrum's seed format is different from BIP39; this example motivates studying the technique, but does not establish the security or deniability of MHFE's optional profile.

## Backward Compatibility

MHFE changes no Bitcoin consensus, network or wallet rules. A suite 3 container is a valid 24-word BIP39 mnemonic, and a suite 4 container a valid mnemonic of the source's own length, 12, 15, 18 or 21 words; software that implements only suite 3 rejects the shorter containers at its 24-word check. Ordinary wallets accept either and derive an unrelated wallet from it; the MHFE workflow must therefore stay separate from ordinary wallet recovery. That unrelated wallet can serve as a decoy only against someone who does not know that MHFE was used, and, like any decoy, only if its balance and history fit what that person knows about the owner. The decoy passwords analysed in the supplement remain possible when the use of MHFE is known. After recovery, the original seed phrase and any BIP39 passphrase work in every BIP39 wallet exactly as before.

## Security Considerations

Unless a statement names suite 4, this section concerns suite 3; suite 4 is covered by its own section. Of the results below, only the analysis of plausible deniability is extended to suite 4, in the supplement.

In suite 3:

- a full recovery to test a password guess against the container of a uniformly generated source that is otherwise unknown, by the verifier of a short source or against public wallet data such as a known address with the empty or a known BIP39 passphrase, uses twelve Argon2id calls; without the optional wallet check, the container of a 24-word source alone confirms nothing;
- in the direct guessing strategy, an Argon2id result is reused between containers when the password candidate and salt are the same;
- with a known source and container, a password filter uses eleven of the twelve calls;
- for a 24-word source with an unknown, independently chosen BIP39 passphrase, with or without the optional wallet check, the pair-search strategy uses twelve Argon2id calls per MHFE password and one BIP39 seed computation per pair.

What these statements rest on:

- **Proved.** The guessing bounds for the first and third strategies are proved in the random-oracle model: the supplement's Theorems 3 and 4 and Corollary G10, which models public wallet data as a further random function of the source entropy, bound an attacker's success probability by that of testing the most likely passwords with twelve, or eleven, Argon2id calls each and guessing one more, plus explicit terms for the cheaper hash computations and, with a known pair, a credit for at most one partly tested password. Attacks that spend twelve, or eleven, calls per password match the bounds up to these terms and the stated losses from false matches. Those losses need not be small for large numbers of tested passwords. For the fourth strategy's two budgets, Theorem 5 of the supplement bounds seed-recovery probability by the total probability of the pairs those budgets allow, plus explicit error terms, under the conditions given in the profile's recommended use; with the empty or a known passphrase it also bounds password recovery for a phrase generated with the wallet check.
- **Open models.** The supplement's open questions include bounds on overall work sharing between several containers, guessing bounds for suite 4, wallet data that depends on an unknown BIP39 passphrase for 12- to 21-word sources, a phrase created with the wallet check when additional data identifies it beyond the target wallet reference allowed by Theorem 5, non-uniform or partly known sources and broader deniability experiments.

The bounds assume the ideal functions specified in the supplement's model. Their applicability to the concrete primitives, including Argon2id's salt separation, needs separate justification; call counts alone do not prove physical memory or time costs. The supplement separates these questions from additional mathematical results and independent validation.

Neither suite provides a built-in verifier for 24-word sources; the optional source profile provides a separate 16-bit filter, with the limitations stated in its section.

These bounds are separate from the analysis of plausible deniability. For one container and one prepared disclosure, the supplement bounds the adversary's advantage in telling a prepared disclosure from an honest one by about the probability of guessing the real password, in stated experiments and, for the general bound, in the random-oracle model. For a freshly uniform 24-word source independent of the shared oracle tables, passwords and the adversary's prior information, everything disclosed, from the container to the disclosed wallet's record, has exactly the same distribution in both cases if the refusal of fixed points and the redrawing of the decoy password are set aside; the general bound also accounts for these two events and for the chance of finding the real wallet.

The guarantee assumes that the adversary does not know that the original seed phrase has fewer than 24 words, that initial passwords are independently drawn from the same oracle-independent distribution and that the decoy wallet's public history follows the same usage scenario as an honest wallet's. Where deniability matters, further copies of a backup should be exact copies of one verified container, and no other record of the phrase that an adversary could find and link to the container, such as a paper copy or a hardware wallet, should contradict the disclosure.

The analysis does not cover wallets linked by transfers or other records, what changes between repeated demands, two different passwords named for one container, several containers of one phrase made with different passwords or settings, a BIP39 passphrase, a leaked password, side channels or whether a particular person will believe the disclosure. The analysis has not been independently reviewed by a cryptographer.

MHFE provides no authentication: anyone can alter a container and recompute its checksum, and the verifier screens wrong passwords and most accidental corruption but does not authenticate the container. A replaced container phrase may be detected during recovery, but confirming the wallet's identity requires comparison with trusted wallet data, whatever the source length. Keeping an independent copy of the container in another place and comparing the copies can reveal differences; checking a known address after recovery confirms the wallet rather than merely a verifier match.

A password shared by several containers is only as safe as the weakest of them: finding it through any one, including through a leaked original seed phrase with its container, opens all of them.

MHFE is deterministic: the same source, password and settings always give the same container. Copies of a backup can therefore be recreated exactly, and identical containers reveal something. Under the same suite, normalized password and settings they come from identical packed states. If the source length is also the same, the original seed phrase is identical. A packed state does not by itself fix the length of the original seed phrase, because the 256 bits of a short source's state are also a valid 24-word entropy. Across different passwords or settings, equal containers establish nothing about the sources. For fixed inputs there is exactly one correct container, so an independent implementation can detect a different result by recomputing it; this checks the result, not whether the software leaked secrets.

If Argon2id or the construction were weakened in future, existing containers could not be upgraded in place, since they carry no version; they would have to be decrypted and encrypted again under a new suite.

The password remains the main protection. Use independently chosen random words, such as dice words from the EFF list [6]; passwords that people make up themselves are usually much weaker than their length suggests. The supplement's attack-cost table compares password choices under explicit assumptions.

Higher settings increase the cost per guess; each extra independently chosen random word multiplies the password search space by 7,776. For storage over decades, allow for guesses becoming cheaper over time. The supplement also covers the threat models, farms and botnets, the composition with the BIP39 passphrase, determinism and side channels.

## Reference Implementation

The Rust library, command-line tool and WebAssembly build `hobby-eng/mhfe` implement suite 3 from version 0.4.0, and suite 4 and the three optional profiles from version 0.5.0, with the reference C implementation of Argon2 as their single Argon2 engine for native and browser builds. Browser builds compile the same reference C code to WebAssembly with Emscripten (or an equivalent toolchain): one build with threads for cross-origin isolated pages and one without threads for all other pages; the build script and its flags are in the reference implementation (`scripts/build-argon2-wasm.sh`). Versions from 0.4.0 on do not support suite 2. The last release that implements suite 2, `v0.3.0`, remains available. Version `v0.6.0` implements the recovery length rules, the 16-bit wallet check of every 24-word reading and the re-encryption rules of this specification. Its rehearsal check still names a single length when several lengths match and none was stated; version 0.6.1 is to report every matching length. Versions 0.4.0 and 0.5.0 still accept a stated length that replaces detection. The suite 3 corpus comes from implementation revision `cc91b0bab58f51c08a3562a5ef441e7faab726b4`, with its recovery cases from revision `3a705db6945b35f8c248c44893e71ddc36902af4`, and the suite 4 corpus matches the files of revision `aedd4cee4301c794af3693b64017083386115adc`.

# Test Vectors

## Suite corpora and conformance

The suite 3 corpus in `vectors/suite3/` contains 17 positive round transcripts, seven negative recovery cases and 54 fast validation cases. The recorded full-cost replay with the independent OpenSSL 3.5.5 Argon2 engine reproduced every positive transcript in both directions and all seven negative cases. One of them, `stated-24-words`, gives the result of the current length rules; `selected-24-words` keeps the result of the removed manual mode, as the corpus notes explain. The fast cases cover password encoding, input validation and verifier serialization without Argon2 work. Companion notes record source, generator and verifier provenance, validation coverage and remaining limits. These results do not constitute an independently authored MHFE implementation or a cryptographic security review.

Published suite 3 vector sets MUST cover every source length, the defaults, a non-zero PIM, memory level 1, a non-zero PIM and memory level together, and a non-ASCII password, with the exact normalized password bytes and every round's salt and mask input messages, salt, Argon2id output, mask and state. They MUST be reproduced in both directions by an independent Argon2 implementation, such as OpenSSL or RustCrypto, with all suite parameters including four lanes. Published vector sets MUST identify the generator and independent verifier revisions.

Conformance cases MUST include invalid checksums and out-of-range values. Each case MUST state its expected outcome under the length rules of Recovering a mnemonic: the readings given, with their word counts, order and labels, or that no reading is given. Wrong-password and wrong-setting cases MUST NOT expect a rejection in every case: unless a short length is stated, such a recovery normally gives the unverified 24-word reading.

Password-encoding cases MUST cover an unassigned code point, a noncharacter, a Private Use character, the rejected characters NUL, TAB, LF, CR, U+0085, U+2028 and U+2029, invalid UTF-8, an unpaired surrogate at a JavaScript boundary, canonical-equivalent spellings and spaces. They MUST also cover exactly 1024 and 1025 normalized UTF-8 bytes, an NFKD expansion that crosses the 1024-byte limit and an NFKD contraction from more than 1024 input bytes to a valid result. The expected normalized bytes or rejection MUST be recorded. Vectors are released under CC0-1.0.

The standard libsodium password-hashing API fixes the lane count at one and cannot reproduce these four-lane vectors.

The suite 4 corpus in `vectors/suite4/` contains 10 positive round transcripts, four recovery cases and 67 fast validation cases. The source record reports a full-cost replay of every transcript and recovery case in both directions with the independent OpenSSL 3.5.5 Argon2 engine; the import recomputed everything except the Argon2id calls from this specification. Its vector sets MUST cover each source length of 12, 15, 18 and 21 words, the defaults, a non-zero PIM, memory level 1, a non-zero PIM and memory level together, a non-ASCII password, the refusal of a 24-word source, a case showing that ENT separates otherwise equal salt and mask inputs, wrong-password and wrong-setting recoveries, and every round's inputs and states in both directions, under the same requirements for independent reproduction as suite 3. Fast conformance cases without Argon2 work MUST show that every combination of supplied suite, supplied length and container length that the recovery table rejects is refused before any Argon2id call.

The archived suite 2 vectors are in `vectors/archive/suite-2/`, apart from the current corpus in `vectors/suite3/`. They remain valid for suite 2 and preserve compatibility checks for that format, but the current implementation no longer replays them.

## Optional wallet check: MHFE-WALLET-CHECK-SEED-1

Positive wallet-check digests and negative passphrase and serialization cases are listed in the profile vectors.

### Repair words: MHFE-REPAIR-1

Repair words for all four card sizes and examples of unreadable and miscopied words are listed in the profile vectors.

### Password check word: MHFE-PASSWORD-CHECK-1

Dice rolls, check indexes, resulting passwords and a missing-word recovery example are listed in the profile vectors.

## Appendix: Suite 2

Suite 2 (`MHFE-BIP39-256-EXPERIMENTAL-2`) is an archived experimental format defined by the specification published as release `v0.3.0`. The following summary is informative; requirements introduced in suite 3 do not apply retroactively to suite 2, and implementations of suite 3 are not required to support suite 2. Release `v0.3.0` of `mhfe` implements it, and its published vectors are listed under archived suite 2. In short, suite 2 differs from suite 3 in its identifier and domain strings, 512 MiB of Argon2id memory with no memory level, PIM $0 \ldots 31$ with the same pass formula, salt and mask messages $\mathrm{DS} \| \mathrm{BE32}(\mathrm{PIM}) \| \mathrm{BE32}(i) \| R$ without $\mathrm{BE32}(\mathrm{MEM})$, and password normalization with Unicode 18.0.0 (UAX #15 revision 58) [46] instead of 17.0.0 [17]. The released text is also kept, marked as historical, in the archive. An optional final-word-preserving profile for suite 2 was drafted and implemented after that release but never released; the archive notes point to its text, and the supplement keeps its analysis as a research idea.

## AI Assistance and Acknowledgments

Sergei Semenov defined the research direction and practical requirements and made the publication decisions. This specification was developed through extended interaction with ChatGPT (OpenAI) and Claude (Anthropic), which contributed substantially to drafting, literature discovery, calculations, counterarguments and adversarial review. It has not received an independent expert cryptographic review.

## Changelog

The version history, including the changes in each release, is kept in `CHANGELOG.md`.

Previous specification releases remain available: v0.5.0, which adds suite 4 and the optional profiles, v0.4.0, which defines suite 3, and v0.3.0, which defines suite 2.

## Copyright

Copyright © 2026 Sergei Semenov. This specification and the repository documentation are licensed under the Creative Commons Attribution 4.0 International License (`CC-BY-4.0`); see `LICENSE`. The test vectors in `vectors/` are released under CC0-1.0, following BIP 3's recommendation [1]. When sharing or adapting this material, credit Sergei Semenov, link to the license and the source, and indicate changes; attribution must not imply endorsement.

## References

References are numbered by first citation in this specification, followed by first citation of additional sources in the supplement. Both documents use this single list and the same numbers.

1. Murch, "Updated BIP Process," BIP 3, ver. 1.4.0, Dec. 9, 2025. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0003.md. [Accessed: Sep. 20, 2026].

2. E. Lombrozo, "BIP Classification," BIP 123, Aug. 26, 2015. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0123.mediawiki. [Accessed: Sep. 20, 2026].

3. M. Palatinus, P. Rusnak, A. Voisine, and S. Bowe, "Mnemonic code for generating deterministic keys," BIP 39, Sep. 10, 2013. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki. [Accessed: Sep. 18, 2026].

4. A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, RFC Editor, Sep. 2021, doi: 10.17487/RFC9106.

5. Cryptosteel, "How to Use Cryptosteel Capsule." [Online]. Available: https://cryptosteel.com/how-to-use-capsule/. [Accessed: Sep. 18, 2026].

6. J. Bonneau, "Deep Dive: EFF's New Wordlists for Random Passphrases," Electronic Frontier Foundation, Jul. 19, 2016. [Online]. Available: https://www.eff.org/deeplinks/2016/07/new-wordlists-random-passphrases; word list: https://www.eff.org/files/2016/07/18/eff_large_wordlist.txt. [Accessed: Oct. 6, 2026].

7. Chick3nman, "Hashcat v6.2.6 benchmark on the Nvidia RTX 4090," benchmark by blazer, GitHub Gist, Oct. 14, 2022. [Online]. Available: https://gist.github.com/Chick3nman/32e662a5bb63bc4f51b847bb422222fd. [Accessed: Sep. 30, 2026].

8. Tarion, "How to encrypt an existing BIP-39 mnemonic with a password without changing the seed?" *Bitcoin Stack Exchange*, May 5, 2021. [Online]. Available: https://bitcoin.stackexchange.com/questions/106036/. [Accessed: Sep. 19, 2026].

9. T. Kaupat, "Encryption of an existing BIP39 mnemonic without changing the seed," *bitcoin-dev mailing list*, May 5, 2021. [Online]. Available: https://gnusha.org/pi/bitcoindev/CAPyCnfvqVT00C2TZ86GXf856jNJqPXY0duRa1CfdCqC0ecC6xA@mail.gmail.com/. [Accessed: Oct. 3, 2026].

10. T. Kaupat, *go-bip39*, GitHub repository, rev. `3ab2b81a7576aedbe1e1a347cab359e383dbf248`, Dec. 17, 2024. [Online]. Available: https://github.com/Niondir/go-bip39/blob/3ab2b81a7576aedbe1e1a347cab359e383dbf248/encryption.go. Relevant earlier revisions: `6615be49f50a990856ec5a65e7b3d9e985644946`, May 5, 2021, and `2a307b8f25e0454ebbe9bbae0fcb7659fafcbba2`, May 10, 2021. [Accessed: Sep. 19, 2026].

11. M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*, NIST SP 800-38A, Dec. 2001, Appendix B, doi: 10.6028/NIST.SP.800-38A.

12. S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," RFC 2119, BCP 14, RFC Editor, Mar. 1997, doi: 10.17487/RFC2119.

13. B. Leiba, "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words," RFC 8174, BCP 14, RFC Editor, May 2017, doi: 10.17487/RFC8174.

14. M.-J. Saarinen and J.-P. Aumasson, "The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)," RFC 7693, RFC Editor, Nov. 2015, doi: 10.17487/RFC7693.

15. National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.

16. H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, RFC Editor, Feb. 1997, doi: 10.17487/RFC2104.

17. K. Whistler, Ed., "Unicode Normalization Forms," Unicode Standard Annex #15, rev. 57, Unicode 17.0.0, Jul. 30, 2025. [Online]. Available: https://www.unicode.org/reports/tr15/tr15-57.html. [Accessed: Sep. 28, 2026].

18. P. Wuille, "Hierarchical Deterministic Wallets," BIP 32, Feb. 11, 2012. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki. [Accessed: Sep. 28, 2026].

19. P. Rusnak, A. Kozlik, O. Vejpustek, T. Susanka, M. Palatinus, and J. Hoenicke, "Shamir's Secret-Sharing for Mnemonic Codes," SLIP-0039, Dec. 18, 2017. [Online]. Available: https://github.com/satoshilabs/slips/blob/master/slip-0039.md. [Accessed: Sep. 21, 2026].

20. M. S. Turan, E. Barker, W. Burr, and L. Chen, *Recommendation for Password-Based Key Derivation, Part 1: Storage Applications*, NIST SP 800-132, Dec. 2010, Section 5.1, doi: 10.6028/NIST.SP.800-132.

21. V. T. Hoang and P. Rogaway, "On Generalized Feistel Networks," in *Advances in Cryptology--CRYPTO 2010*, LNCS 6223. Berlin, Germany: Springer, 2010, pp. 613-630, doi: 10.1007/978-3-642-14623-7_33. Full version with appendices: Cryptology ePrint Archive, Paper 2010/301, rev. Nov. 29, 2018. [Online]. Available: https://eprint.iacr.org/2010/301. [Accessed: Oct. 2, 2026].

22. X. Lai and J. L. Massey, "A Proposal for a New Block Encryption Standard," in *Advances in Cryptology--EUROCRYPT '90*, LNCS 473. Berlin, Germany: Springer, 1991, pp. 389-404, doi: 10.1007/3-540-46877-3_35.

23. S. Vaudenay, "On the Lai-Massey Scheme," in *Advances in Cryptology--ASIACRYPT '99*, LNCS 1716. Berlin, Germany: Springer, 1999, pp. 8-19, doi: 10.1007/978-3-540-48000-6_2.

24. V. T. Hoang, B. Morris, and P. Rogaway, "An Enciphering Scheme Based on a Card Shuffle," in *Advances in Cryptology--CRYPTO 2012*, LNCS 7417. Berlin, Germany: Springer, 2012, pp. 1-13, doi: 10.1007/978-3-642-32009-5_1.

25. A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications," in *2016 IEEE European Symposium on Security and Privacy*. Piscataway, NJ, USA: IEEE, 2016, pp. 292-302, doi: 10.1109/EuroSP.2016.31.

26. Password Hashing Competition, "Password Hashing Competition and our recommendation for hashing passwords: Argon2." [Online]. Available: https://www.password-hashing.net/. [Accessed: Sep. 29, 2026].

27. OWASP Cheat Sheet Series, "Password Storage Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: Sep. 29, 2026].

28. J. Patarin, "Luby-Rackoff: 7 Rounds Are Enough for 2^{n(1-epsilon)} Security," in *Advances in Cryptology--CRYPTO 2003*, LNCS 2729. Berlin, Germany: Springer, 2003, pp. 513-529, doi: 10.1007/978-3-540-45146-4_30.

29. J. Patarin, "Security of Random Feistel Schemes with 5 or More Rounds," in *Advances in Cryptology--CRYPTO 2004*, LNCS 3152. Berlin, Germany: Springer, 2004, pp. 106-122, doi: 10.1007/978-3-540-28628-8_7.

30. M. Luby and C. Rackoff, "How to Construct Pseudorandom Permutations from Pseudorandom Functions," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 373-386, Apr. 1988, doi: 10.1137/0217022.

31. J. Black and P. Rogaway, "Ciphers with Arbitrary Finite Domains," in *Topics in Cryptology-- CT-RSA 2002*, LNCS 2271. Berlin, Germany: Springer, 2002, pp. 114-130, doi: 10.1007/3-540-45760-7_9.

32. M. Caldwell and A. Voisine, "Passphrase-protected private key," BIP 38, Nov. 20, 2012. [Online]. Available: https://github.com/bitcoin/bips/blob/master/bip-0038.mediawiki. [Accessed: Sep. 21, 2026].

33. JonDerThan, *MnemonicCrypt*, GitHub repository, rev. `d3c9315b483805689fd978b7dc783b0e86676473`, Oct. 31, 2025. [Online]. Available: https://github.com/JonDerThan/mnemonic-crypt/tree/d3c9315b483805689fd978b7dc783b0e86676473. [Accessed: Sep. 20, 2026].

34. kklash, *Mnemonikey*, GitHub repository, rev. `0bd15d84d23ffd7e439eb3217c4215dd8df8894f`, Jan. 29, 2024. [Online]. Available: https://github.com/kklash/mnemonikey/tree/0bd15d84d23ffd7e439eb3217c4215dd8df8894f. [Accessed: Sep. 21, 2026].

35. C. J. DeLisle, *pktseed*, GitHub repository, rev. `1ec6b87f6603579bac8b63f38d73710ee8e36425`, Aug. 28, 2021. [Online]. Available: https://github.com/cjdelisle/pktseed/tree/1ec6b87f6603579bac8b63f38d73710ee8e36425. [Accessed: Sep. 22, 2026].

36. B. Matthews, *seed-otp*, GitHub repository, rev. `70b51e05daf054355bd7691188ff7720afc7ca3c`, Apr. 30, 2021. [Online]. Available: https://github.com/brndnmtthws/seed-otp/tree/70b51e05daf054355bd7691188ff7720afc7ca3c. [Accessed: Sep. 18, 2026].

37. mifunetoshiro, *Seedshift*, GitHub repository, rev. `853423930e29b388ff936f581d9b692944319d46`, Jul. 26, 2025. [Online]. Available: https://github.com/mifunetoshiro/Seedshift/tree/853423930e29b388ff936f581d9b692944319d46. [Accessed: Sep. 18, 2026].

38. mifunetoshiro, *bip39_obfuscator*, GitHub repository, rev. `0d82f4809fe4bec0e53d4487a3dd9e34142af04b`, Oct. 25, 2021. [Online]. Available: https://github.com/mifunetoshiro/bip39_obfuscator/tree/0d82f4809fe4bec0e53d4487a3dd9e34142af04b. [Accessed: Sep. 19, 2026].

39. EnteroPositivo, *BIP39Colors*, GitHub repository, rev. `df3bc100416d8acc48d7cad02050e5eb3ac177ae`, Jul. 15, 2023. [Online]. Available: https://github.com/EnteroPositivo/bip39colors/tree/df3bc100416d8acc48d7cad02050e5eb3ac177ae. [Accessed: Sep. 22, 2026].

40. The Monero Project, *monero*, GitHub repository, rev. `160e21504aed2a9b6dfdba0970517161383b04e5`, Oct. 2, 2026, `encrypt_key` and `decrypt_key` in src/cryptonote_basic/cryptonote_format_utils.cpp. [Online]. Available: https://github.com/monero-project/monero/blob/160e21504aed2a9b6dfdba0970517161383b04e5/src/cryptonote_basic/cryptonote_format_utils.cpp. [Accessed: Oct. 3, 2026].

41. tevador, *polyseed*, GitHub repository, rev. `56f634647d4f75596de20a6259b0cf1933949fdc`, Sep. 24, 2026, README.md, section "Checksum", and `polyseed_crypt` in src/polyseed.c. [Online]. Available: https://github.com/tevador/polyseed/tree/56f634647d4f75596de20a6259b0cf1933949fdc. [Accessed: Oct. 3, 2026].

42. T. Hardin, *seed-encrypt*, GitHub repository, rev. `cda9b158a2859e1e1c077fb05b10fe0f83fab988`, Sep. 5, 2026; first commit Sep. 12, 2024. [Online]. Available: https://github.com/Tyler-Hardin/seed-encrypt/tree/cda9b158a2859e1e1c077fb05b10fe0f83fab988. [Accessed: Oct. 3, 2026].

43. A. Juels and T. Ristenpart, "Honey Encryption: Security Beyond the Brute-Force Bound," in *Advances in Cryptology--EUROCRYPT 2014*, LNCS 8441. Berlin, Germany: Springer, 2014, pp. 293-310, doi: 10.1007/978-3-642-55220-5_17.

44. R. Canetti, C. Dwork, M. Naor, and R. Ostrovsky, "Deniable Encryption," in *Advances in Cryptology--CRYPTO '97*, LNCS 1294. Berlin, Germany: Springer, 1997, pp. 90-104, doi: 10.1007/BFb0052229.

45. The Electrum developers, "Electrum Seed Version System," *Electrum documentation*, sections "Seed generation" and "Security implications." [Online]. Available: https://electrum.readthedocs.io/en/latest/seedphrase.html. [Accessed: Oct. 5, 2026].

46. K. Whistler, Ed., "Unicode Normalization Forms," Unicode Standard Annex #15, rev. 58, Unicode 18.0.0, Aug. 12, 2026. [Online]. Available: https://www.unicode.org/reports/tr15/tr15-58.html. [Accessed: Sep. 22, 2026].

47. M. Bellare, T. Ristenpart, and S. Tessaro, "Multi-Instance Security and Its Application to Password-Based Cryptography," in *Advances in Cryptology--CRYPTO 2012*, LNCS 7417. Berlin, Germany: Springer, 2012, pp. 312-329, doi: 10.1007/978-3-642-32009-5_19. Full version: Cryptology ePrint Archive, Paper 2012/196. [Online]. Available: https://eprint.iacr.org/2012/196. [Accessed: Oct. 14, 2026].

48. G. Demay, P. Gaži, U. Maurer, and B. Tackmann, "Per-Session Security: Password-Based Cryptography Revisited," in *Computer Security--ESORICS 2017*, LNCS 10492. Cham, Switzerland: Springer, 2017, pp. 408-426, doi: 10.1007/978-3-319-66402-6_24.

49. V. Shoup, "Sequences of Games: A Tool for Taming Complexity in Security Proofs," Cryptology ePrint Archive, Paper 2004/332, 2004. [Online]. Available: https://eprint.iacr.org/2004/332. [Accessed: Sep. 30, 2026].

50. A. Czeskis, D. J. St. Hilaire, K. Koscher, S. D. Gribble, T. Kohno, and B. Schneier, "Defeating Encrypted and Deniable File Systems: TrueCrypt v5.1a and the Case of the Tattling OS and Applications," in *3rd USENIX Workshop on Hot Topics in Security (HotSec 08)*, Jul. 2008. [Online]. Available: https://www.usenix.org/legacy/event/hotsec08/tech/full_papers/czeskis/czeskis.pdf. [Accessed: Oct. 1, 2026].

51. J. Patarin, "Security of balanced and unbalanced Feistel Schemes with Linear Non Equalities," Cryptology ePrint Archive, Paper 2010/293, May 18, 2010. [Online]. Available: https://eprint.iacr.org/2010/293. [Accessed: Sep. 21, 2026].

52. J. Patarin, "Generic Attacks on Feistel Schemes," in *Advances in Cryptology--ASIACRYPT 2001*, LNCS 2248. Berlin, Germany: Springer, 2001, pp. 222-238, doi: 10.1007/3-540-45682-1_14. Extended version: Cryptology ePrint Archive, Paper 2008/036.

53. L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC '96)*, 1996, pp. 212-219, doi: 10.1145/237814.237866.

54. J. Proos and C. Zalka, "Shor's Discrete Logarithm Quantum Algorithm for Elliptic Curves," *Quantum Information and Computation*, vol. 3, no. 4, pp. 317-344, 2003. [Online]. Available: https://arxiv.org/abs/quant-ph/0301141. [Accessed: Oct. 1, 2026].

55. National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, updated May 9, 2023, doi: 10.6028/NIST.FIPS.197-upd1.

56. crocket, "Offline Transaction Signing," guest tutorial, *Monero Docs*, section "Creating a new offline wallet with seed offset passphrase." [Online]. Available: https://docs.getmonero.org/cold-storage/offline-transaction-signing/#creating-a-new-offline-wallet-with-seed-offset-passphrase. [Accessed: Oct. 3, 2026].

57. Lightning Labs, *lnd*, GitHub repository, rev. `f3a8f4e8ae8237ff2b40e4724de45085170df962`, Oct. 1, 2026, aezeed/README.md; package added in pull request #773, Mar. 2, 2018. [Online]. Available: https://github.com/lightningnetwork/lnd/blob/f3a8f4e8ae8237ff2b40e4724de45085170df962/aezeed/README.md. [Accessed: Oct. 3, 2026].

58. Coinkite, "Seed XOR," *Coldcard firmware*, GitHub repository, rev. `3e32e32a49c551b35e7bb3cecbf910241f553537`, Sep. 30, 2026, docs/seed-xor.md. [Online]. Available: https://github.com/Coldcard/firmware/blob/3e32e32a49c551b35e7bb3cecbf910241f553537/docs/seed-xor.md. [Accessed: Oct. 3, 2026].

59. G. Tonoski, *BIP39-XOR*, GitHub repository, rev. `d08b4daf8d83768e2dbd7c5db9225a3f5de6c943`, Jul. 27, 2025. [Online]. Available: https://github.com/GregTonoski/BIP39-XOR/tree/d08b4daf8d83768e2dbd7c5db9225a3f5de6c943. [Accessed: Oct. 3, 2026].

60. vrpxfv, *PhraseCrypt*, GitHub repository, rev. `f9331a69d86a75b146d372f9e5174ae8f9ad04b2`, Sep. 2, 2026. [Online]. Available: https://github.com/vrpxfv/PhraseCrypt/tree/f9331a69d86a75b146d372f9e5174ae8f9ad04b2. [Accessed: Oct. 3, 2026].

61. M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods for Format-Preserving Encryption*, NIST SP 800-38G, updated Aug. 4, 2016, doi: 10.6028/NIST.SP.800-38G. The second public draft of Revision 1 was published Feb. 3, 2025; it is not a final publication.

62. B. Morris, H. Oberschelp, and H. S. Santhakumar, "Format Preserving Encryption in the Bounded Retrieval Model," arXiv:2307.08158, Jul. 16, 2023, doi: 10.48550/arXiv.2307.08158.

63. B. Morris, P. Rogaway, and T. Stegers, "How to Encipher Messages on a Small Domain," in *Advances in Cryptology--CRYPTO 2009*, LNCS 5677. Berlin, Germany: Springer, 2009, pp. 286-302, doi: 10.1007/978-3-642-03356-8_17.

64. H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, RFC Editor, May 2010, doi: 10.17487/RFC5869. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869.html. [Accessed: Oct. 5, 2026].

65. G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Oblivious Key-Value Stores and Amplification for Private Set Intersection," in *Advances in Cryptology--CRYPTO 2021*, Part II, LNCS 12826. Cham, Switzerland: Springer, 2021, pp. 395-425, doi: 10.1007/978-3-030-84245-1_14. Public version, Sections 2.1-2.2: [Online]. Available: https://iacr.org/archive/crypto2021/12826253/12826253.pdf. [Accessed: Oct. 5, 2026].

66. R. Anderson, R. Needham, and A. Shamir, "The Steganographic File System," in *Information Hiding, Second International Workshop, IH'98*, LNCS 1525. Berlin, Germany: Springer, 1998, pp. 73-82, doi: 10.1007/3-540-49380-8_6.

67. M. Bellare and P. Rogaway, "The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs," in *Advances in Cryptology--EUROCRYPT 2006*, LNCS 4004. Berlin, Germany: Springer, 2006, pp. 409-426, doi: 10.1007/11761679_25.

68. Y. Dodis, T. Ristenpart, J. Steinberger, and S. Tessaro, "To Hash or Not to Hash Again? (In)differentiability Results for H^2 and HMAC," in *Advances in Cryptology--CRYPTO 2012*, LNCS 7417. Berlin, Germany: Springer, 2012, pp. 348-366, doi: 10.1007/978-3-642-32009-5_21.

69. M. Mahmoody, T. Moran, and S. Vadhan, "Time-Lock Puzzles in the Random Oracle Model," in *Advances in Cryptology--CRYPTO 2011*, LNCS 6841. Berlin, Germany: Springer, 2011, pp. 39-50, doi: 10.1007/978-3-642-22792-9_3.

70. B. Cohen and K. Pietrzak, "Simple Proofs of Sequential Work," in *Advances in Cryptology--EUROCRYPT 2018*, LNCS 10821. Cham, Switzerland: Springer, 2018, pp. 451-467, doi: 10.1007/978-3-319-78375-8_15.

71. F. F. Yao and Y. L. Yin, "Design and Analysis of Password-Based Key Derivation Functions," in *Topics in Cryptology--CT-RSA 2005*, LNCS 3376. Berlin, Germany: Springer, 2005, pp. 245-261, doi: 10.1007/978-3-540-30574-3_17.

72. M. Naor and O. Reingold, "On the Construction of Pseudorandom Permutations: Luby-Rackoff Revisited," *Journal of Cryptology*, vol. 12, no. 1, pp. 29-66, 1999, doi: 10.1007/PL00003817.

73. S. Coretti, Y. Dodis, S. Guo, and J. Steinberger, "Random Oracles and Non-Uniformity," in *Advances in Cryptology--EUROCRYPT 2018*, LNCS 10820. Cham, Switzerland: Springer, 2018, pp. 227-258, doi: 10.1007/978-3-319-78381-9_9. Full version: Cryptology ePrint Archive, Paper 2017/937. [Online]. Available: https://eprint.iacr.org/2017/937. [Accessed: Oct. 14, 2026].

74. P. Farshim and S. Tessaro, "Password Hashing and Preprocessing," in *Advances in Cryptology--EUROCRYPT 2021*, LNCS 12697. Cham, Switzerland: Springer, 2021, pp. 64-91, doi: 10.1007/978-3-030-77886-6_3.

75. J. Alwen and B. Tackmann, "Moderately Hard Functions: Definition, Instantiations, and Applications," in *Theory of Cryptography--TCC 2017*, LNCS 10677. Cham, Switzerland: Springer, 2017, pp. 493-526, doi: 10.1007/978-3-319-70500-2_17.

76. C. Dodd, P. Farshim, S. F. Shahandashti, and K. Southern, "Multi-Instance Unrecoverability of iMHF-Based Password Hashing," in *Computer Security--ESORICS 2026*. Cham, Switzerland: Springer, 2026, pp. 44-64, doi: 10.1007/978-3-032-38708-0_3.

77. G. Demay, P. Gaži, U. Maurer, and B. Tackmann, "Query-Complexity Amplification for Random Oracles," in *Information Theoretic Security--ICITS 2015*, LNCS 9063. Cham, Switzerland: Springer, 2015, pp. 159-180, doi: 10.1007/978-3-319-17470-9_10.

78. T. Isobe and K. Shibutani, "Generic Key Recovery Attack on Feistel Scheme," in *Advances in Cryptology--ASIACRYPT 2013*, LNCS 8269. Berlin, Germany: Springer, 2013, pp. 464-485, doi: 10.1007/978-3-642-42033-7_24.

79. P. Gaži and U. Maurer, "Cascade Encryption Revisited," in *Advances in Cryptology--ASIACRYPT 2009*, LNCS 5912. Berlin, Germany: Springer, 2009, pp. 37-51, doi: 10.1007/978-3-642-10366-7_3.

80. G. Zaverucha, "Stronger Password-Based Encryption Using All-or-Nothing Transforms," Microsoft Research, Tech. Rep. MSR-TR-2015-63, 2015. [Online]. Available: https://www.microsoft.com/en-us/research/publication/stronger-password-based-encryption-using-all-or-nothing-transforms/. [Accessed: Oct. 14, 2026].

81. A. Biryukov and D. Khovratovich, "Egalitarian Computing," in *25th USENIX Security Symposium*, Aug. 2016, pp. 315-326. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/biryukov. [Accessed: Oct. 14, 2026].

82. N. Dedić, G. Itkis, L. Reyzin, and S. Russell, "Upper and Lower Bounds on Black-Box Steganography," *Journal of Cryptology*, vol. 22, no. 3, pp. 365-394, Jul. 2009, doi: 10.1007/s00145-008-9020-3.

83. M. Scarlata, M. Backendal, and M. Haller, "MFKDF: Multiple Factors Knocked Down Flat," in *33rd USENIX Security Symposium*, Aug. 2024, pp. 4301-4318. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/scarlata. [Accessed: Oct. 15, 2026].